

Mizar Evolution: **なぜ今、一階述語論理なのか**
自動証明・数学的記述・検証可能な計算の再接続
presenter notes edition

中正 和久

岩手県立大学

TPP 2026、**理化学研究所 AIP 東京オフィス**、2026 年 11 月 16–17 日

Mizar Evolution: なぜ今、一階述語論理なのか

Mizar Evolution: なぜ今、一階述語論理なのか
自動証明・数学的記述・検証可能な計算の再接続
presenter notes edition

中正 和久

岩手県立大学

TPP 2026、理化学研究所 AIP 東京オフィス、2026 年 11 月 16-17 日

Read aloud:

- Mizar は証明検査系です。一階の集合論の上に、人間が読める数学の言葉で書かれた 50 年分のライブラリを持っています。
- Mizar Evolution、略して Mizar Evo は、その言語と道具を設計し直すプロジェクトです。今日は機能を並べません。自動証明器をめぐる一つの謎から始めて、なぜ論理を一階のままにするのかを説明します。
- 話す内容にはすべて、事実・研究仮説・将来構想のラベルを付けます。

1.1 - 類似した成功率、異なる評価条件

- 報告された成功率は近いが、評価条件は異なる。
- 評価単位・前提選択・時間予算が異なるため、成功率の直接比較はできない。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 1. Two Hammers, Two Numbers

└ 1.1 - 類似した成功率、異なる評価条件

1.1 - 類似した成功率、異なる評価条件

	MizAR 60 (JTP 2023)	Sledgehammer on AFP (CICM 2015)
報告された成功率	58.4% を証明	60.7% を証明
評価単位	MML 1147 の top-level theorem と lemma (57,897 件)	AFP 開発内部の proof goal (6,934 件)
前提	ライブラリ全体から学習的に選択	開発内から MePo フィルタで選択
時間予算	ポートフォリオ合計 420 CPU 秒	各 prover 30 秒、4 prover union を oracle として信頼。一行再構成は各 prover 約 50 %
成功の判定	ATP 証明の発見 (hammering mode)	

- 報告された成功率は近いが、評価条件は異なる。
- 評価単位・前提選択・時間予算が異なるため、成功率の直接比較はできない。

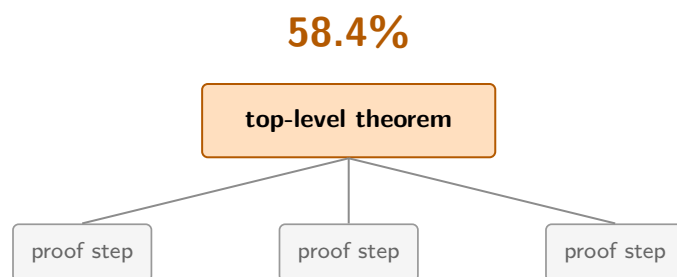
Read aloud:

- 報告された成功率は近いが、評価条件は異なる。
- 評価単位・前提選択・時間予算が異なるため、成功率の直接比較はできない。

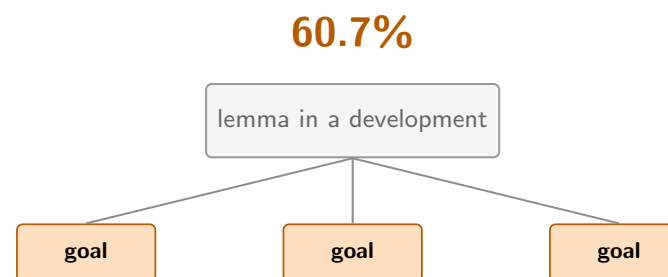
Optional notes and sources:

- 「ハンマー」は goal を自動証明器に送る道具。「前提」は prover が使ってよい事実。
- Source: Jakubův et al. 2023, results 1-3; Blanchette et al. 2015, Section "Proof Automation with Sledgehammer", Figure 13.
- MizAR の 75% は、人または機械がライブラリから前提を選んだ条件。質問用にとっておく。

1.2 - 評価単位: 定理全体と局所ゴール



MizAR 60 (ITP 2023) counts *whole top-level theorems and lemmas* 57,897 in MML 1147; premises from the whole library; 420 CPU s portfolio



Sledgehammer on the AFP (CICM 2015) counts *proof goals as they arise* 6,934 goals from 128 theories; 30 s per prover; union trusted as oracle

Same-looking percentages. Different units, different premise regimes, different budgets.

- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

Mizar Evolution: なぜ今、一階述語論理なのか

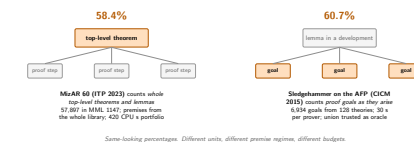
└ Part 1. Two Hammers, Two Numbers

└ 1.2 - 評価単位: 定理全体と局所ゴール

Read aloud:

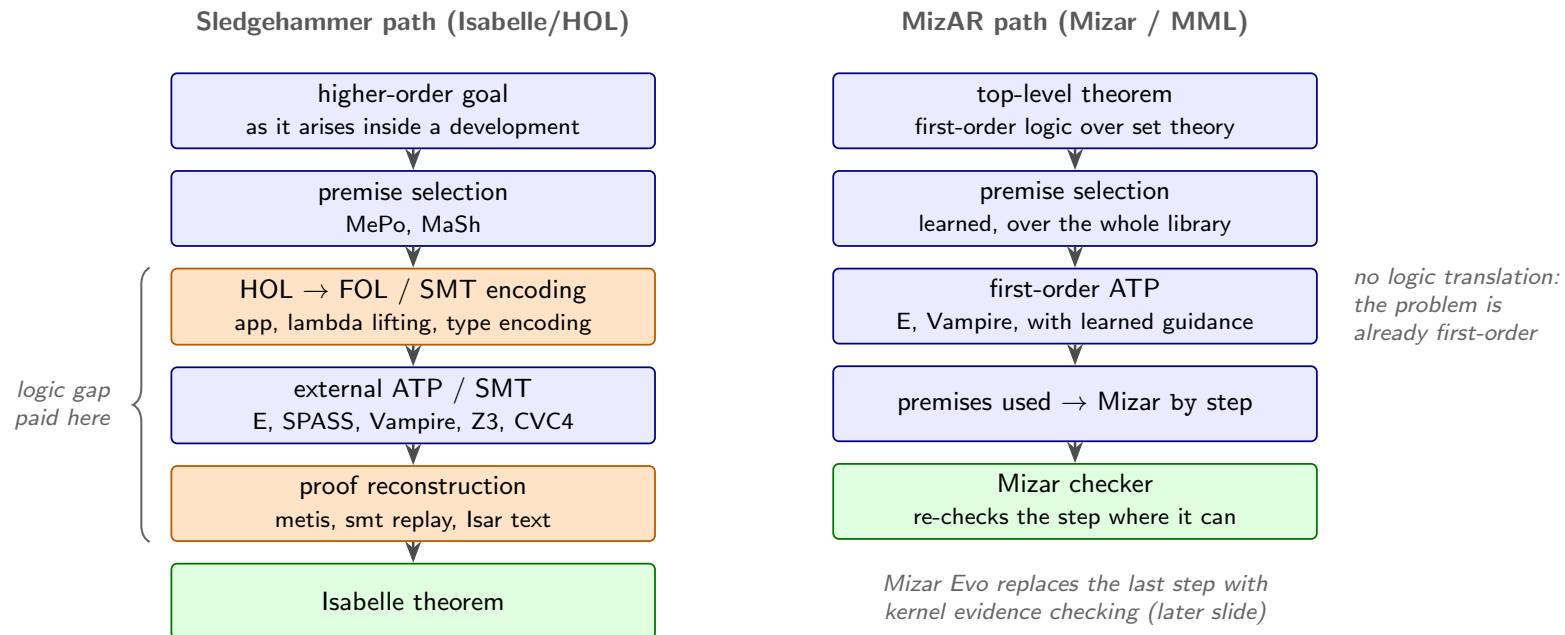
- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

1.2 - 評価単位: 定理全体と局所ゴール



- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

1.3 - 一階 ATP への二つの接続経路



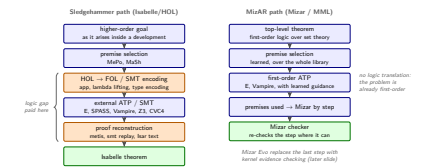
- Sledgehammer: 高階のゴールを一階論理または SMT の問題へ翻訳し、得られた証明を Isabelle 内で再構成する。
- MizAR: 一階の問題を ATP へ送る。検査系の論理と ATP の問題表現との距離が短い。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 1. Two Hammers, Two Numbers

└ 1.3 - 一階 ATP への二つの接続経路

1.3 - 一階 ATP への二つの接続経路



- Sledgehammer: 高階のゴールを一階論理または SMT の問題へ翻訳し、得られた証明を Isabelle 内で再構成する。
- MizAR: 一階の問題を ATP へ送る。検査系の論理と ATP の問題表現との距離が短い。

Read aloud:

- **Sledgehammer: 高階のゴールを一階論理または SMT の問題へ翻訳し、得られた証明を Isabelle 内で再構成する。**
- **MizAR: 一階の問題を ATP へ送る。検査系の論理と ATP の問題表現との距離が短い。**

Optional notes and sources:

- **どちらも既存システムの説明** (Blanchette, Kaliszyk, Paulson, Urban 2016; Jakubův et al. 2023)。オレンジの箱が翻訳と再構成の層。
- MizAR の見出しの数字は ATP 証明を数える。Mizar checker は、推論が検査器の強さの範囲なら、得られた by ステップを再検査する。

1.4 - 研究課題: 評価条件と設計の影響

評価条件の差か、設計上の差か

- **研究仮説:** 一階ライブラリでは、検査系の論理と ATP の問題表現との距離が短い。高階処理系では、翻訳と再構成に追加のコストが生じる。
- **事実:** Sledgehammer は、高階処理系における一階 ATP の有効性を示した。
- **研究仮説:** 高階処理系と翻訳を組み合わせた設計が最適かは未検証。比較対象となる現代的な一階 ITP が少ない。
- **Mizar Evo:** 現代的な検査系・ライブラリ・ATP による仮説検証のための基盤。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 1. Two Hammers, Two Numbers

└ 1.4 - 研究課題: 評価条件と設計の影響

1.4 - 研究課題: 評価条件と設計の影響

評価条件の差か、設計上の差か

- 研究仮説: 一階ライブラリでは、検査系の論理と ATP の問題表現との距離が短い。高階処理系では、翻訳と再構成に追加のコストが生じる。
- 事実: Sledgehammer は、高階処理系における一階 ATP の有効性を示した。
- 研究仮説: 高階処理系と翻訳を組み合わせた設計が最適かは未検証。比較対象となる現代的な一階 ITP が少ない。
- Mizar Evo: 現代的な検査系・ライブラリ・ATP による仮説検証のための基盤。

Read aloud:

- 評価条件の差か、設計上の差か
- 研究仮説: 一階ライブラリでは、検査系の論理と ATP の問題表現との距離が短い。高階処理系では、翻訳と再構成に追加のコストが生じる。
- 事実: Sledgehammer は、高階処理系における一階 ATP の有効性を示した。
- 研究仮説: 高階処理系と翻訳を組み合わせた設計が最適かは未検証。比較対象となる現代的な一階 ITP が少ない。
- Mizar Evo: 現代的な検査系・ライブラリ・ATP による仮説検証のための基盤。

2.1 - 高階論理における関数

sketch

```
f :: 'a => 'b      x :: 'a  
  
P (f x)
```

- HOL: 関数型と関数適用を論理に組み込む。
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- 高階の構造を用いた簡潔な数学的記述が可能。
- E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 2. Where Complexity Lives

└ 2.1 - 高階論理における関数

Read aloud:

- HOL の文の概形.
- **HOL: 関数型と関数適用を論理に組み込む。**
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- **高階の構造を用いた簡潔な数学的記述が可能。**
- **E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。**

2.1 - 高階論理における関数

HOL の文の概形 **sketch** (la belle/HOL 風の記法)

$f :: 'a \rightarrow 'b$ $x :: 'a$

$P (f\ x)$

- HOL: 関数型と関数適用を論理に組み込む。
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- 高階の構造を用いた簡潔な数学的記述が可能。
- E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。

2.2 - ATP 接続に伴う符号化と再構成

sketch

```
F X          -> app(F, X)
(%x. t) ...   -> fresh constant + defining axioms   (lambda lifting)
polymorphic types -> type guards or type tags
Boolean-valued terms -> extra encoding
```

- 関数変数の適用は `app` で明示。ラムダ抽象は `lambda lifting` またはコンビネータで、型は `guard` または `tag` で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : `metis`、`smt` の `replay`、生成した `Isar` テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。一階の基盤では高階構造の符号化は不要。
- HOL の記述上の利点に対し、ATP 接続時に符号化・再構成のコストが生じる。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 2. Where Complexity Lives

└ 2.2 - ATP 接続に伴う符号化と再構成

2.2 - ATP 接続に伴う符号化と再構成

符号化の手順の概形

$\lambda x. t$	$\rightarrow$	<code>app(λ, x)</code>
$(\lambda x. t) \dots$	$\rightarrow$	<code>fresh constant + defining axioms (lambda lifting)</code>
polymorphic types	$\rightarrow$	<code>type guards or type tags</code>
Boolean-valued terms	$\rightarrow$	<code>extra encoding</code>

- 関数変数の適用は `app` で明示。ラムダ抽象は `lambda lifting` またはコンビネータで、型は `guard` または `tag` で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : `metis`, `smt` の `replay`、生成した `Isar` テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。一階の基盤では高階構造の符号化は不要。
- HOL の記述上の利点に対し、ATP 接続時に符号化・再構成のコストが生じる。

Read aloud:

- 符号化の手順の概形.
- 関数変数の適用は `app` で明示。ラムダ抽象は `lambda lifting` またはコンビネータで、型は `guard` または `tag` で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : `metis`, `smt` の `replay`、生成した `Isar` テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。一階の基盤では高階構造の符号化は不要。
- HOL の記述上の利点に対し、ATP 接続時に符号化・再構成のコストが生じる。

Optional notes and sources:

- Source: Meng and Paulson 2008 (高階節から一階節への翻訳) ; Blanchette, Böhme, Popescu, Smallbone 2016 (型の符号化) ; Blanchette, Kaliszyk, Paulson, Urban 2016 (サーベイ) ; Schurr, Fleury, Desharnais 2021 (再構成)。
- Sledgehammer が実際に使う符号化は `prover` とオプションで変わる。この枚は概形。

2.3 - 集合論における関数: Mizar の基礎

exact MML excerpt

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

exact MML excerpt

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- 関数は対の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 2. Where Complexity Lives

└ 2.3 - 集合論における関数: Mizar の基礎

2.3 - 集合論における関数: Mizar の基礎

関数適用は定義された集合論的關係 exact MML excerpt

```
func f.x -> set means
:Def2:
[x,it] in f if x in dom f otherwise it = {};
```

型付き関数は部分関数の上の soft type exact MML excerpt

```
definition
let X,Y;
mode Function of X,Y is quasi_total PartFunc of X,Y;
end;
```

- 関数は対の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

Read aloud:

- 関数適用は定義された集合論的關係.
- 型付き関数は部分関数の上の soft type.
- 関数は対の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

Optional notes and sources:

- Source (2026 年 10 月 7 日確認) :
https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz (138-140 行、
 FUNCT_1:def 2)、https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz
 (87-90 行)。GPL-3.0-or-later / CC-BY-SA-3.0-or-later。
- quasi_total (FUNCT_2 の 36-44 行) は、Y が空でなければ定義域が X 全体であることを言う。

2.4 - Mizar の言語設計: 集合論の細部を抽象化

sketch

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- Mizar Evo は、Mizar が 50 年にわたり蓄積した言語設計を継承・拡張する。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 2. Where Complexity Lives

└ 2.4 - Mizar の言語設計: 集合論の細部を抽象化

Read aloud:

- ソース上の関数表現.
- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- Mizar Evo は、Mizar が 50 年にわたり蓄積した言語設計を継承・拡張する。

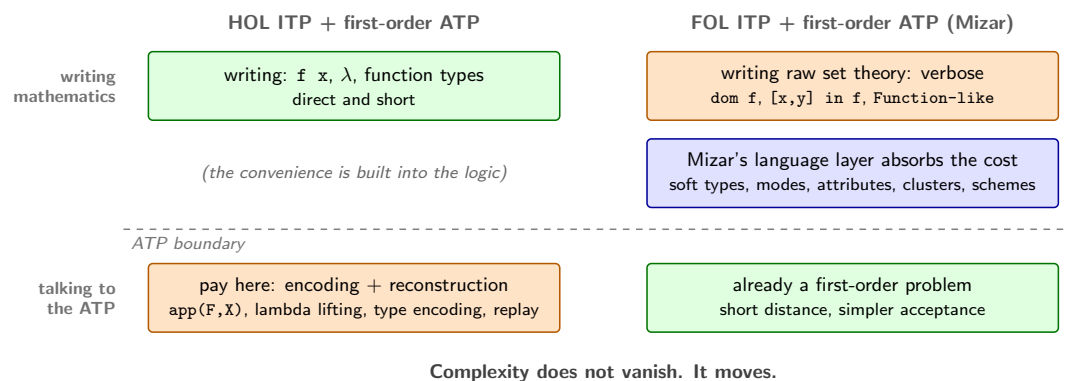
2.4 - Mizar の言語設計: 集合論の細部を抽象化

ソース上の関数表現 **sketch** (現行 Mizar と Mizar Evo で有効)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- Mizar Evo は、Mizar が 50 年にわたり蓄積した言語設計を継承・拡張する。

2.5 - 複雑さを担う層の違い



HOL と FOL では、複雑さを担う層が異なる。

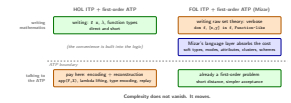
- HOL: ATP 接続時の符号化・再構成。FOL: 集合論の直接記述に伴う煩雑さを、Mizar の言語機構で吸収。
- Mizar Evo の設計方針: 一階の基盤論理を保ち、記述上の複雑さを言語層で抽象化する。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 2. Where Complexity Lives

└ 2.5 - 複雑さを担う層の違い

2.5 - 複雑さを担う層の違い



HOL と FOL では、複雑さを担う層が異なる。

- HOL: ATP 接続時の符号化・再構成。FOL: 集合論の直接記述に伴う煩雑さを、Mizar の言語機構で吸収。
- Mizar Evo の設計方針: 一階の基盤論理を保ち、記述上の複雑さを言語層で抽象化する。

Read aloud:

- HOL と FOL では、複雑さを担う層が異なる。
- HOL: ATP 接続時の符号化・再構成。FOL: 集合論の直接記述に伴う煩雑さを、Mizar の言語機構で吸収。
- Mizar Evo の設計方針: 一階の基盤論理を保ち、記述上の複雑さを言語層で抽象化する。

- この表は設計上の解釈を示す。実測による評価は Mizar Evo の 2027 年の課題。

2026-10-09

Mizar Evolution: なぜ今、一階述語論理なのか
└ Part 2. Where Complexity Lives

└ 2.6 - 設計上のトレードオフ

deep dive

Read aloud:

- この表は設計上の解釈を示す。実測による評価は Mizar Evo の 2027 年の課題。

2.6 - 設計上のトレードオフ

	HOL ITP + ATP	FOL ITP + ATP
記述	高階機能による簡潔な記述	直接記述すると冗長
関数	基本的な高階の対象	一階の集合論的对象
ATP 接続	符号化が必要	距離が短い
再構成	論理表現の差を埋める必要	原理的には単純
言語層の役割	HOL の抽象	一階の細部の抽象化

• この表は設計上の解釈を示す。実測による評価は Mizar Evo の 2027 年の課題。

3.1 - 論理は保ち、言語を現代化する

数学的記述の層を継承し、
処理系と開発基盤を再構築する。

- **一階論理と Tarski-Grothendieck 集合論を基盤として維持。**
- **soft type、mode、attribute、registration、structure、宣言的証明を継承。**
- **暗黙の選択を明示化し、汎用化機構を統一。自動化の追跡と、現代的なコンパイラ構成を導入する。**
- **現代化の要点: template、algorithm、信頼境界、開発基盤。**

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.1 - 論理は保ち、言語を現代化する

3.1 - 論理は保ち、言語を現代化する

数学的記述の層を継承し、
処理系と開発基盤を再構築する。

- 一階論理と Tarski-Grothendieck 集合論を基盤として維持。
- soft type、mode、attribute、registration、structure、宣言的証明を継承。
- 暗黙の選択を明示化し、汎用化機構を統一。自動化の追跡と、現代的なコンパイラ構成を導入する。
- 現代化の要点: template、algorithm、信頼境界、開発基盤。

Read aloud:

- 数学的記述の層を継承し、 処理系と開発基盤を再構築する。
- 一階論理と Tarski-Grothendieck 集合論を基盤として維持。
- soft type、mode、attribute、registration、structure、宣言的証明を継承。
- 暗黙の選択を明示化し、汎用化機構を統一。自動化の追跡と、現代的なコンパイラ構成を導入する。
- 現代化の要点: template、algorithm、信頼境界、開発基盤。

3.2 - Template: 一階論理上の汎用的な数学的記述

template はパラメータ付きの definition ブロック specification example

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

- 型・値・述語・関数子をパラメータとして扱う。従来の Mizar の scheme も、述語パラメータ付き定理として統合する。
- 基盤論理に無制限の二階量化は追加しない。各インスタンス化を検査し、一階の証明義務を生成する。
- 汎用的な数学的記述は、表層言語の機構として実現する。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.2 - Template: 一階論理上の汎用的な数学的記述

3.2 - Template: 一階論理上の汎用的な数学的記述

template はパラメータ付きの definition ブロック specification example

```
definition
  let T be type;
  struct MagmaStr [T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

- 型・値・述語・関数子をパラメータとして扱う。従来の Mizar の scheme も、述語パラメータ付き定理として統合する。
- 基盤論理に無制限の二階量化は追加しない。各インスタンス化を検査し、一階の証明義務を生成する。
- 汎用的な数学的記述は、表層言語の機構として実現する。

Read aloud:

- **template はパラメータ付きの definition ブロック。**
- **型・値・述語・関数子をパラメータとして扱う。従来の Mizar の scheme も、述語パラメータ付き定理として統合する。**
- **基盤論理に無制限の二階量化は追加しない。各インスタンス化を検査し、一階の証明義務を生成する。**
- **汎用的な数学的記述は、表層言語の機構として実現する。**

Optional notes and sources:

- Source: doc/spec/en/18.templates.md, sections 18.1-18.2 and 18.8.

specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: defpred に続けて by NatInduction[P], Base, Step。
- 関数子パラメータは集合ではなく schema レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (PermProduct[T]、qua によるビュー)、および本資料 Backup 4。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.3 - Scheme を template に統合

deep dive

3.3 - Scheme を template に統合

deep dive

述語パラメータ付き定理としての帰納法

specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: defpred に続けて by NatInduction[P], Base, Step。
- 関数子パラメータは集合ではなく schema レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (PermProduct [T], qua によるビュー)、および本資料 Backup 4。

Read aloud:

- 述語パラメータ付き定理としての帰納法。
- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: defpred に続けて by NatInduction[P], Base, Step。
- 関数子パラメータは集合ではなく schema レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (PermProduct [T], qua によるビュー)、および本資料 Backup 4。

3.4 - Algorithm: アルゴリズムの推論と検証可能な計算

- **algorithm の目的は、アルゴリズムに関する推論と検証可能な計算。高階関数の代替とは異なる役割を担う。**
- **一階論理では、完全な推論系を背景に resolution、superposition、saturation による自動探索が発展。これは歴史的傾向であり、論理的必然ではない。**
- **LCF/HOL 系では、tactic と tactical によるプログラム可能な証明構成が発展。**
- **Mizar は宣言的証明と組込みの自動化を採用。ユーザ定義の tactic 言語は持たない。**
- **Mizar Evo の algorithm は、処理系が検証する契約付き手続き。単なる tactic 言語の追加ではない。**

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.4 - Algorithm: アルゴリズムの推論と検証可能な計算

3.4 - Algorithm: アルゴリズムの推論と検証可能な計算

- algorithm の目的は、アルゴリズムに関する推論と検証可能な計算。高階関数の代替とは異なる役割を担う。
- 一階論理では、完全な推論系を背景に resolution、superposition、saturation による自動探索が進展。これは歴史的傾向であり、論理的必然ではない。
- LCF/HOL 系では、tactic と tactical によるプログラム可能な証明構成が進展。
- Mizar は宣言的証明と組込みの自動化を採用。ユーザ定義の tactic 言語は持たない。
- Mizar Evo の algorithm は、処理系が検証する契約付き手続き。単なる tactic 言語の追加ではない。

Read aloud:

- algorithm の目的は、アルゴリズムに関する推論と検証可能な計算。高階関数の代替とは異なる役割を担う。
- 一階論理では、完全な推論系を背景に resolution、superposition、saturation による自動探索が進展。これは歴史的傾向であり、論理的必然ではない。
- LCF/HOL 系では、tactic と tactical によるプログラム可能な証明構成が進展。
- Mizar は宣言的証明と組込みの自動化を採用。ユーザ定義の tactic 言語は持たない。
- Mizar Evo の algorithm は、処理系が検証する契約付き手続き。単なる tactic 言語の追加ではない。

3.5 - Algorithm: 契約、証明、計算

契約付きのユークリッドの互除法 specification example

```
terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end;
  return x;
end;
```

- 契約・不変条件・停止性の測度から一階の証明義務を生成し、定理と同じ枠組みで検査する。
- 二つの役割: 検証済みの自動化手続きと、by computation による検査済みの計算。仕様化済み。MVM 実行とコード抽出は今後の課題。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.5 - Algorithm: 契約、証明、計算

3.5 - Algorithm: 契約、証明、計算

契約付きのユークリッドの互除法

specification example

```

terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end;
  return x;
end;

```

- 契約・不変条件・停止性の測度から一階の証明義務を生成し、定理と同じ枠組みで検査する。
- 二つの役割: 検証済みの自動化手続きと、by computation による検査済みの計算。仕様化済み。MVM 実行とコード抽出は今後の課題。

Read aloud:

- 契約付きのユークリッドの互除法.
- 契約・不変条件・停止性の測度から一階の証明義務を生成し、定理と同じ枠組みで検査する。
- 二つの役割: 検証済みの自動化手続きと、by computation による検査済みの計算。仕様化済み。MVM 実行とコード抽出は今後の課題。

Optional notes and sources:

- Source: doc/spec/en/20.algorithm_and_verification.md, section 20.12 (外側の definition ブロックと let a, b be Nat; を省略)。

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- 長期的な対象: 暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。
- algorithm は、証明の自動化手続きと応用アルゴリズムの検証を共通の枠組みに結び付ける。対象の拡大は将来構想。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.6 - Algorithm の応用範囲 (将来構想)

deep dive

3.6 - Algorithm の応用範囲 (将来構想)

deep dive

将来構想 (現在の対応範囲には含まれない) :

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- 長期的な対象: 暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。

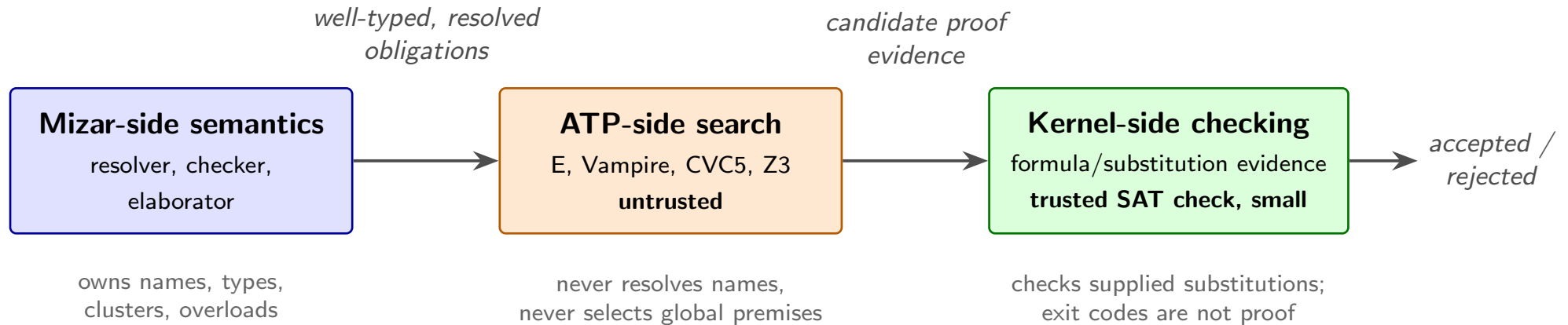
共通する検証・実行の流れ: 手続き・契約の記述 → 不変条件・停止性の指定 → 証明義務の生成 → ATP による証明と kernel 検査 → 具体的な入力での実行 → コード抽出 (将来)。

- **algorithm** は、証明の自動化手続きと応用アルゴリズムの検証を共通の枠組みに結び付ける。対象の拡大は将来構想。

Read aloud:

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- 長期的な対象: 暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。
- 手続き・契約の記述 → 不変条件・停止性の指定 → 証明義務の生成 → ATP による証明と kernel 検査 → 具体的な入力での実行 → コード抽出 (将来)。
- **algorithm** は、証明の自動化手続きと応用アルゴリズムの検証を共通の枠組みに結び付ける。対象の拡大は将来構想。

3.7 - 証明探索と信頼できる検査の分離



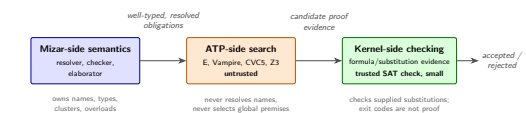
- 一階 ATP は証明探索を担う。探索結果は、信頼できる検査器による検証を要する。
- Mizar 側: 名前・型・cluster・オーバーロードの解決。ATP: 証明探索。kernel: 論理式と代入を、小規模な信頼できる SAT 検査で検証し、受理を判定。
- ATP の終了コードのみでは証明を受理しない。一階自動推論の導入によって信頼基盤を拡大しない設計。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.7 - 証明探索と信頼できる検査の分離

3.7 - 証明探索と信頼できる検査の分離



- 一階 ATP は証明探索を担う。探索結果は、信頼できる検査器による検証を要する。
- Mizar 側: 名前・型・cluster・オーバーロードの解決。ATP: 証明探索。kernel: 論理式と代入を、小規模な信頼できる SAT 検査で検証し、受理を判定。
- ATP の終了コードのみでは証明を受理しない。一階自動推論の導入によって信頼基盤を拡大しない設計。

Read aloud:

- 一階 ATP は証明探索を担う。探索結果は、信頼できる検査器による検証を要する。
- Mizar 側: 名前・型・cluster・オーバーロードの解決。ATP: 証明探索。kernel: 論理式と代入を、小規模な信頼できる SAT 検査で検証し、受理を判定。
- ATP の終了コードのみでは証明を受理しない。一階自動推論の導入によって信頼基盤を拡大しない設計。

Optional notes and sources:

- Source: doc/design/architecture/en/08.reasoning_boundary.md; Białystok 資料 Story 4; 本資料 Backup 5 に evidence の中身。

3.8 - 50 年の蓄積を踏まえた開発基盤の現代化

- Mizar の数学的思想を継承し、ソフトウェア構成は現代の開発要件に合わせて再設計する。
- 言語とともに開発基盤全体を現代化。各機能の詳細は Białystok 資料を参照。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 3. Modernizing Mizar's Answer

└ 3.8 - 50 年の蓄積を踏まえた開発基盤の現代化

3.8 - 50 年の蓄積を踏まえた開発基盤の現代化

MML の 50 年の蓄積から見えた課題	Mizar Evo
article が依存の単位	明示的 import を持つモジュール
グローバル名前管理	namespace、完全修飾名
配布とバージョン管理が弱い	package、SemVer、lock file
フルビルド	依存指紋による差分ビルド
ATP は外付け、自動化が見えにくい	第一級の ATP パイプライン、解決トレース、kernel evidence
IDE 連携と機械可読な入出力が弱い	LSP、構造化診断、エージェント向けインタフェース

- Mizar の数学的思想を継承し、ソフトウェア構成は現代の開発要件に合わせて再設計する。
- 言語とともに開発基盤全体を現代化。各機能の詳細は Białystok 資料を参照。

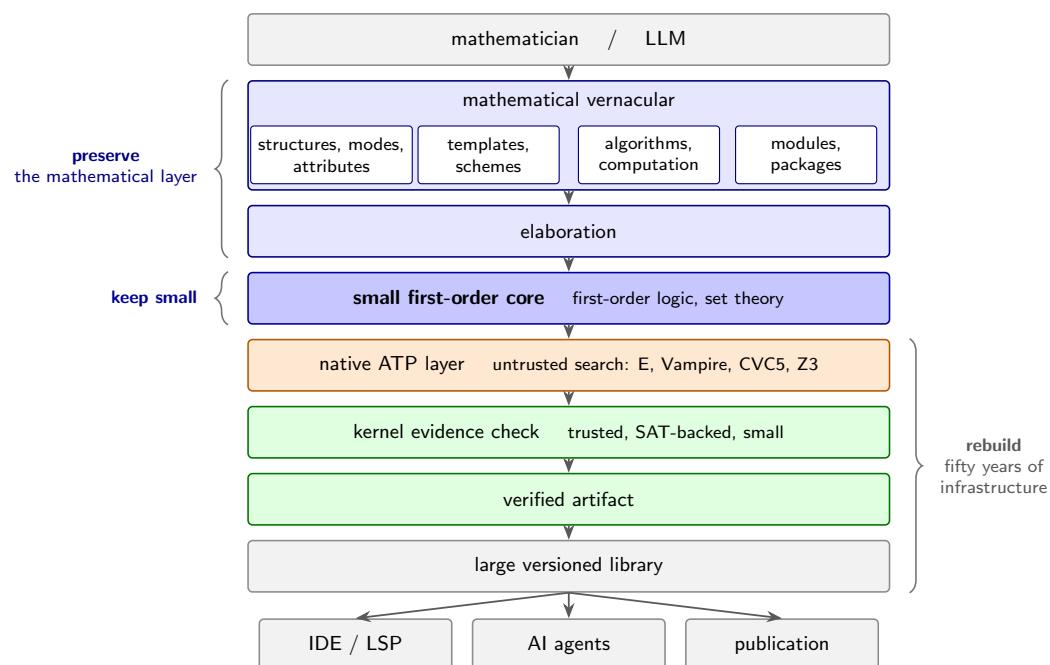
Read aloud:

- Mizar の数学的思想を継承し、ソフトウェア構成は現代の開発要件に合わせて再設計する。
- 言語とともに開発基盤全体を現代化。各機能の詳細は Białystok 資料を参照。

Optional notes and sources:

- 批判ではない。50 年前には一般的でなかったソフトウェア工学を、形式数学の環境に持ち込む話。
- 詳細: Białystok 資料 Story 1, 3, 5, 8; 本資料 Backup 6-8。

4.1 - 全体像



豊かな数学的記述を、
小規模な一階論理の基盤と現代的な開発環境で支える。

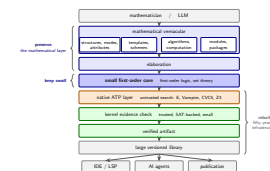
2026-10-09

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 4. The AI Era

└ 4.1 - 全体像

4.1 - 全体像

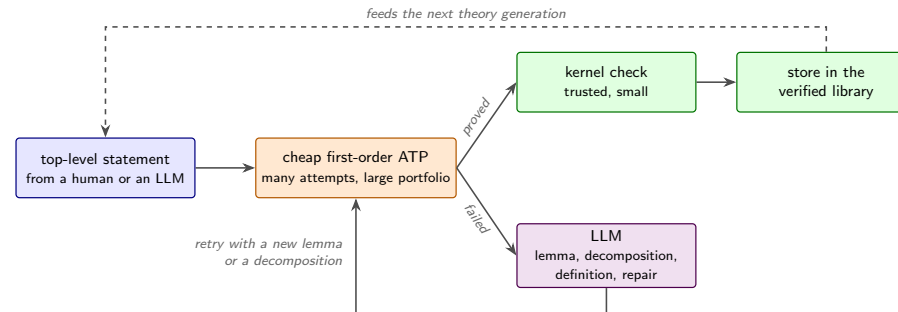


豊かな数学的記述を、
小規模な一階論理の基盤と現代的な開発環境で支える。

Read aloud:

- 豊かな数学的記述を、 小規模な一階論理の基盤と現代的な開発環境で支える。

4.2 - LLM • ATP • Mizar Evo の役割分担



LLM thinks. ATP proves. Mizar Evo remembers and verifies.
division of labor: design intent — cost-effectiveness of the cascade: research hypothesis

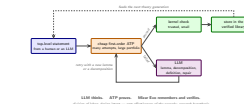
- **LLM: 理論・定義・証明方針・補題の生成と失敗回復。ATP: 低コストで反復可能な一階証明探索。Mizar Evo: 数学的表現、検証済みライブラリ、信頼できる検査、来歴の管理。**
- **研究仮説: 数学の大量生成に伴い、ATP による低コストな証明探索の価値が高まる。このループの費用対効果は未評価。**
- **現在の仕様: ATP の入力引用は引用された前提と局所仮定に限定。ライブラリ全体を対象とするハンマーは 2027 年の研究課題。**

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 4. The AI Era

└ 4.2 - LLM • ATP • Mizar Evo の役割分担

4.2 - LLM • ATP • Mizar Evo の役割分担



- LLM: 理論・定義・証明方針・補題の生成と失敗回復。ATP: 低コストで反復可能な一階証明探索。Mizar Evo: 数学的表現、検証済みライブラリ、信頼できる検査、来歴の管理。
- 研究仮説: 数学の大量生成に伴い、ATP による低コストな証明探索の価値が高まる。このループの費用対効果は未評価。
- 現在の仕様: ATP の入力には引用された前提と局所仮定に限定。ライブラリ全体を対象とするハンマーは 2027 年の研究課題。

Read aloud:

- **LLM: 理論・定義・証明方針・補題の生成と失敗回復。ATP: 低コストで反復可能な一階証明探索。Mizar Evo: 数学的表現、検証済みライブラリ、信頼できる検査、来歴の管理。**
- **研究仮説: 数学の大量生成に伴い、ATP による低コストな証明探索の価値が高まる。このループの費用対効果は未評価。**
- **現在の仕様: ATP の入力には引用された前提と局所仮定に限定。ライブラリ全体を対象とするハンマーは 2027 年の研究課題。**

Optional notes and sources:

- Source: `doc/spec/en/21.source_code_annotation_and_atp.md`, section 21.7.2, item 4 (グローバルライブラリからの自動前提選択は無し) ;
`doc/design/architecture/en/21.ai_agent_interface.md` (編集クラス)。

4.3 - 仕様と実装の状況 (2026 年 10 月)

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド (字句解析・構文解析・構文木)、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 4. The AI Era

└ 4.3 - 仕様と実装の状況 (2026 年 10 月)

4.3 - 仕様と実装の状況 (2026 年 10 月)

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド (字句解析・構文解析・構文木)、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

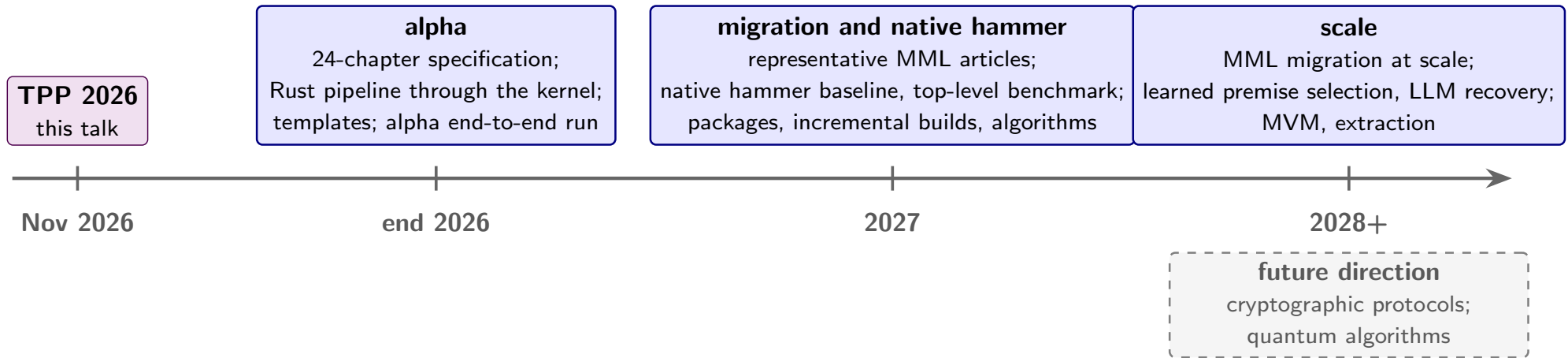
Read aloud:

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド (字句解析・構文解析・構文木)、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

Optional notes and sources:

- Source: [doc/design/todo.md](#), Crate Status (2026 年 10 月 7 日時点)。講演前に再確認。

5.1 - ロードマップ



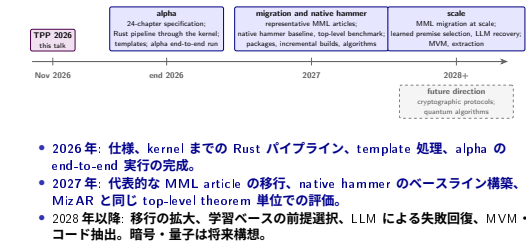
- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行の完成。
- 2027 年: 代表的な MML article の移行、native hammer のベースライン構築、MizAR と同じ top-level theorem 単位での評価。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM・コード抽出。暗号・量子は将来構想。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 5. Roadmap And Closing

└ 5.1 - ロードマップ

5.1 - ロードマップ



Read aloud:

- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行の完成。
- 2027 年: 代表的な MML article の移行、native hammer のベースライン構築、MizAR と同じ top-level theorem 単位での評価。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM・コード抽出。暗号・量子は将来構想。

5.2 - 一階自動推論を設計原理とする意義

高階処理系での一階 ATP の活用は確立している。
研究課題は、一階自動推論を当初から設計原理とすることで、
何が可能になるか。

1. 一階論理と集合論を基盤として維持。
2. Mizar の言語機構で一階の細部を抽象化。
3. template により汎用的な数学的記述を拡張。
4. algorithm により検証可能な計算を統合。
5. 50 年の蓄積を踏まえ、ソフトウェア基盤を再構築。
6. LLM と ATP を、それぞれの得意分野に応じて連携。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 5. Roadmap And Closing

└ 5.2 - 一階自動推論を設計原理とする意義

5.2 - 一階自動推論を設計原理とする意義

高階処理系での一階 ATP の活用は確立している。
研究課題は、一階自動推論を当初から設計原理とすることで、
何が可能になるか。

Mizar Evo の六つの設計方針:

1. 一階論理と集合論を基盤として維持。
2. Mizar の言語機構で一階の細部を抽象化。
3. template により汎用的な数学的記述を拡張。
4. algorithm により検証可能な計算を統合。
5. 50 年の蓄積を踏まえ、ソフトウェア基盤を再構築。
6. LLM と ATP を、それぞれの得意分野に応じて連携。

Read aloud:

- 高階処理系での一階 ATP の活用は確立している。研究課題は、一階自動推論を当初から設計原理とすることで、何が可能になるか。
- 一階論理と集合論を基盤として維持。
- Mizar の言語機構で一階の細部を抽象化。
- template により汎用的な数学的記述を拡張。
- algorithm により検証可能な計算を統合。
- 50 年の蓄積を踏まえ、ソフトウェア基盤を再構築。
- LLM と ATP を、それぞれの得意分野に応じて連携。

5.3 - おわりに

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- 基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。
- 議論したい点: 2027 年のベンチマークで、評価対象・単位・条件をそろえた比較をどう設計するか。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Part 5. Roadmap And Closing

└ 5.3 - おわりに

5.3 - おわりに

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- 基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。
- 議論したい点: 2027 年のベンチマークで、評価対象・単位・条件をそろえた比較をどう設計するか。

Read aloud:

- **Keep the foundation small. Keep the mathematics readable. Modernize everything else.**
- **基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。**
- **議論したい点: 2027 年のベンチマークで、評価対象・単位・条件をそろえた比較をどう設計するか。**

Backup 1. MizAR 60 の詳細

- **データ:** MPTP で出力した MML 1147、無名の top-level lemma を含む 57,897 件の定理。MizAR 40 と同一版を用いた比較。
- top-level lemma の 58.4% を large-theory (hammering) mode で証明。ユーザの前提指定なし、ポートフォリオ合計 420 CPU 秒 (MizAR 40 は約 40.6%)。
- 人または機械がライブラリから前提を選べる条件では 75% 超 (MizAR 40 は 56%)。
- 最高性能の単一手法: hammering mode で 30 秒 40%。人間の前提指定ありで 120 秒 60%。
- 転移: 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- 手法: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 1. MizAR 60 の詳細

└ Backup 1. MizAR 60 の詳細

Backup 1. MizAR 60 の詳細

Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

- データ: MPTP で出力した MML 1147、無名の top-level lemma を含む 57,897 件の定理。MizAR 40 と同一版を用いた比較。
- top-level lemma の 58.4% を large-theory (hammering) mode で証明。ユーザの前提指定なし、ポートフォリオ合計 420 CPU 秒 (MizAR 40 は約 40.6%)。
- 人または機械がライブラリから前提を選べる条件では 75% 超 (MizAR 40 は 56%)。
- 最高性能の単一手法: hammering mode で 30 秒 40%。人間の前提指定ありで 120 秒 60%。
- 転移: 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- 手法: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

Read aloud:

- データ: MPTP で出力した MML 1147、無名の top-level lemma を含む 57,897 件の定理。MizAR 40 と同一版を用いた比較。
- top-level lemma の 58.4% を large-theory (hammering) mode で証明。ユーザの前提指定なし、ポートフォリオ合計 420 CPU 秒 (MizAR 40 は約 40.6%)。
- 人または機械がライブラリから前提を選べる条件では 75% 超 (MizAR 40 は 56%)。
- 最高性能の単一手法: hammering mode で 30 秒 40%。人間の前提指定ありで 120 秒 60%。
- 転移: 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- 手法: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

Optional notes and sources:

- Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

Backup 2. Sledgehammer 評価の詳細

- **設定: 無作為に選んだ AFP の 128 theory、各 100 goal まで。Isabelle2014。MePo フィルタ。各 prover 30 秒をスライスに分割。再構成の制限時間は 2 秒。**
- **複数 prover の結果の和集合を oracle として信頼した場合、goal の 60.7% を証明。**
- **Judgement Day (2010) : 7 theory の 1,240 subgoal、E・SPASS・Vampire を 30 秒で 46%。2015 年の予備評価では 6 prover で 75%。**
- **theory ごとの成功率は 10% – 100%。Sledgehammer は Judgement Day を用いて調整されてきた。**

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 2. Sledgehammer 評価の詳細

└ Backup 2. Sledgehammer 評価の詳細

Backup 2. Sledgehammer 評価の詳細

Prover	One-line / + Isar / + Oracle (6,934 goal に対する %)
E	49.7 / 51.4 / 52.5
SPASS	49.4 / 50.5 / 52.0
Vampire	49.5 / 51.0 / 51.8
Z3	49.6 / 50.0 / 53.7

- 設定: 無作為に選んだ AFP の 128 theory、各 100 goal まで。Isabelle2014。MePo フィルタ。各 prover 30 秒をスライスに分割。再構成の制限時間は 2 秒。
- 複数 prover の結果の和集合を oracle として信頼した場合、goal の 60.7% を証明。
- Judgement Day (2010) : 7 theory の 1,240 subgoal、E・SPASS・Vampire を 30 秒で 46%。2015 年の予備評価では 6 prover で 75%。
- theory ごとの成功率は 10% – 100%。Sledgehammer は Judgement Day を用いて調整されてきた。

Read aloud:

- 設定: 無作為に選んだ AFP の 128 theory、各 100 goal まで。Isabelle2014。MePo フィルタ。各 prover 30 秒をスライスに分割。再構成の制限時間は 2 秒。
- 複数 prover の結果の和集合を oracle として信頼した場合、goal の 60.7% を証明。
- Judgement Day (2010) : 7 theory の 1,240 subgoal、E・SPASS・Vampire を 30 秒で 46%。2015 年の予備評価では 6 prover で 75%。
- theory ごとの成功率は 10% – 100%。Sledgehammer は Judgement Day を用いて調整されてきた。

Optional notes and sources:

- Source: Blanchette, Haslbeck, Matichuk, Nipkow, Mining the Archive of Formal Proofs, CICM 2015, Section "Proof Automation with Sledgehammer", Figure 13; Böhme and Nipkow, IJCAR 2010.

Backup 3. HOL から FOL への符号化と再構成

- 関数適用: 関数変数の適用を `app(F, X)` で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象: `lambda lifting` は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を `guard`、`tag`、または単相化で符号化。選択は健全性・完全性・`prover` 性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による `metis`、SMT 証明の `smt replay`、生成した `Isar` テキスト。再構成の失敗は別途集計。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 3. HOL から FOL への符号化と再構成

└ Backup 3. HOL から FOL への符号化と再構成

Backup 3. HOL から FOL への符号化と再構成

- 関数適用: 関数変数の適用を $\text{app}(F, X)$ で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象: λ lifting は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を guard 、 tag 、または単相化で符号化。選択は健全性・完全性・ prover 性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による metis 、SMT 証明の smt replay 、生成した Isar テキスト。再構成の失敗は別途集計。

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Read aloud:

- 関数適用: 関数変数の適用を $\text{app}(F, X)$ で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象: λ lifting は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を guard 、 tag 、または単相化で符号化。選択は健全性・完全性・ prover 性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による metis 、SMT 証明の smt replay 、生成した Isar テキスト。再構成の失敗は別途集計。

Optional notes and sources:

- Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

specification example

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

specification example

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

└ Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

Read aloud:

- 有界な型パラメータと汎用的な定理.
- インスタンス化.
- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

Optional notes and sources:

- Source: doc/spec/en/18.templates.md, section 18.2.2.

Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

有界な型パラメータと汎用的な定理 specification example (証明は省略)

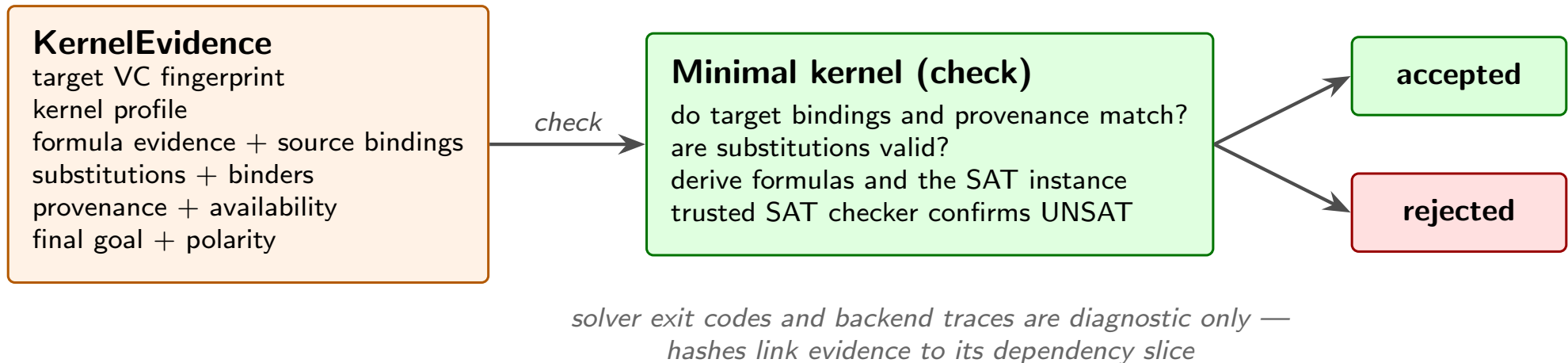
```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

インスタンス化 specification example (必要な registration がある前提)

```
PermProduct [commutative associative unital AddMagma]
PermProduct [commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

Backup 5. Kernel evidence と信頼できる SAT 検査



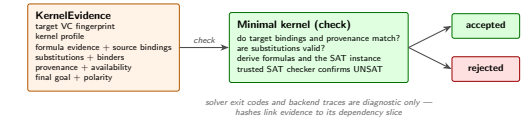
- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 5. Kernel evidence と信頼できる SAT 検査

└ Backup 5. Kernel evidence と信頼できる SAT 検査

Backup 5. Kernel evidence と信頼できる SAT 検査



- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

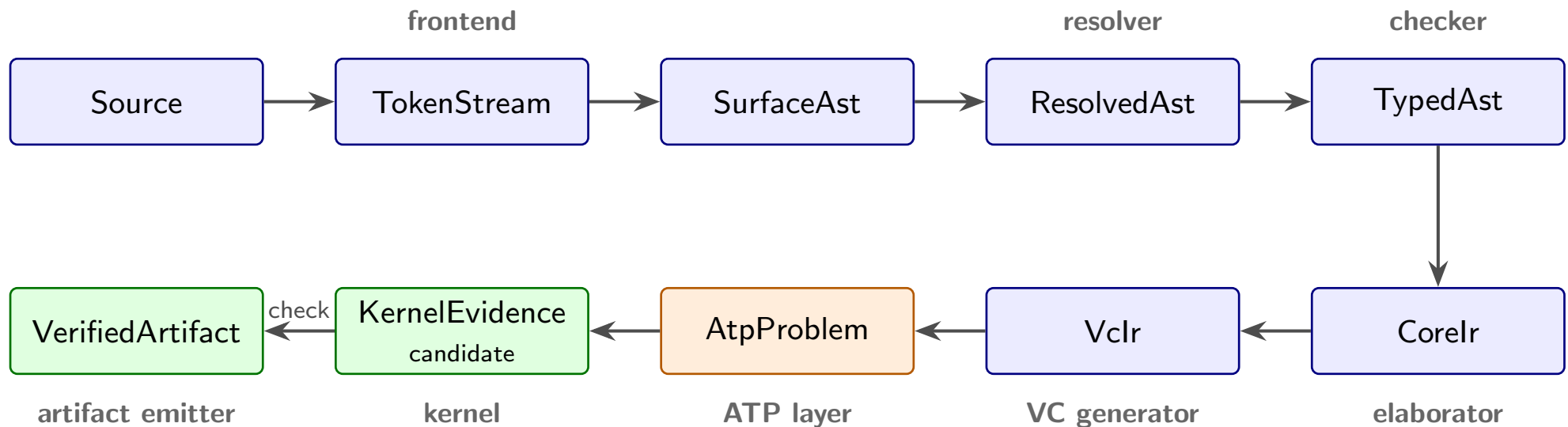
Read aloud:

- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

Optional notes and sources:

- Source: doc/design/architecture/en/08.reasoning_boundary.md, 15.kernel_certificate_format.md.

Backup 6. ATP の中心経路



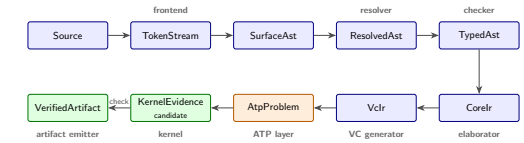
- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 6. ATP の中心経路

└ Backup 6. ATP の中心経路

Backup 6. ATP の中心経路



- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

Read aloud:

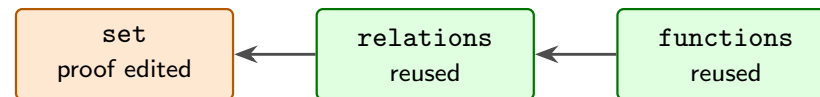
- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

Optional notes and sources:

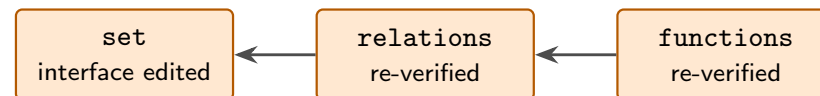
- Source: [doc/design/architecture/en/00.pipeline_overview.md](#); Białystok 資料 Part 10。

Backup 7. 差分検証

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

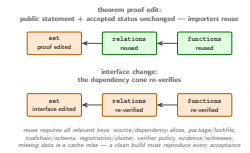
- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 7. 差分検証

└ Backup 7. 差分検証

Backup 7. 差分検証



- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

Read aloud:

- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

Optional notes and sources:

- Source: `doc/design/architecture/en/11.artifact_and_incremental_build.md`, `18.dependency_fingerprint.md`; Białystok 資料 Story 5。

Backup 8. Package manifest

specification example

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- 再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。
- バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 8. Package manifest

└ Backup 8. Package manifest

Backup 8. Package manifest

```
Mizar Evo specification example
[package]
name = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "~1.0"
topology = { version = "~0.9", features = ["metric"] }
```

- 再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。
- バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。

Source: doc/spec/en/23.package_management_and_build_system.md; Białystok 資料 Story 1。

Read aloud:

- 再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。
- バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。

Optional notes and sources:

- Source: doc/spec/en/23.package_management_and_build_system.md; Białystok 資料 Story 1。

Backup 9. 45 分版での追加

1. 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
2. Mizar の関数表現: Function of X, Y 、 $f.x$ 、集合論的対象としての soft type。
3. Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
4. Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
5. Structure とビュー: AddMagma、MulMagma、Magma (Białystok Story 2)。
6. 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
7. 信頼境界: ATP 探索、KernelEvidence、検査された受理 (Backup 5)。

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 9. 45 分版での追加

└ Backup 9. 45 分版での追加

Backup 9. 45 分版での追加

構成を維持し、以下の順で具体例を詳述:

1. 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
2. Mizar の関数表現: Function of X, Y 、 $f.x$ 、集合論的対象としての soft type。
3. Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
4. Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
5. Structure とビュー: AddMagma, MulMagma, Magma (Białystok Story 2)。
6. 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
7. 信頼境界: ATP 探索、KernelEvidence、検査された受理 (Backup 5)。

Read aloud:

- 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
- Mizar の関数表現: Function of X, Y 、 $f.x$ 、集合論的対象としての soft type。
- Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
- Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
- Structure とビュー: AddMagma、MulMagma、Magma (Białystok Story 2)。
- 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
- 信頼境界: ATP 探索、KernelEvidence、検査された受理 (Backup 5)。

Backup 10. 出典と帰属

- FUNCT_1: `https://mizar.uwb.edu.pl/version/current/mml/funct\_1.miz`
- FUNCT_2: `https://mizar.uwb.edu.pl/version/current/mml/funct\_2.miz`
- **MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。 article 名・URL・行番号を発表者ノートに記載。**
- **ベンチマークの数値は Backup 1-2 と references.bib を参照。最終版の作成前に、出版社の情報で書誌事項を確認。**

Mizar Evolution: なぜ今、一階述語論理なのか

└ Backup 10. 出典と帰属

└ Backup 10. 出典と帰属

Backup 10. 出典と帰属

本講演における MML の原文引用:

目的	出典	行	使用箇所
関数適用 $f.x$	<code>funct_1.miz</code>	138-140	Frame 2.3
soft type としての $\text{Function of } X,Y$	<code>funct_2.miz</code>	87-90	Frame 2.3

Source URLs:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

帰属についての注記:

- MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。article 名・URL・行番号を発表者ノートに記載。
- ベンチマークの数値は Backup 1-2 と `references.bib` を参照。最終版の作成前に、出版社の情報で書誌事項を確認。

Read aloud:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz
- MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。article 名・URL・行番号を発表者ノートに記載。
- ベンチマークの数値は Backup 1-2 と `references.bib` を参照。最終版の作成前に、出版社の情報で書誌事項を確認。